

Introduction to Scientific Computing

ENGR1050 – Lecture 23

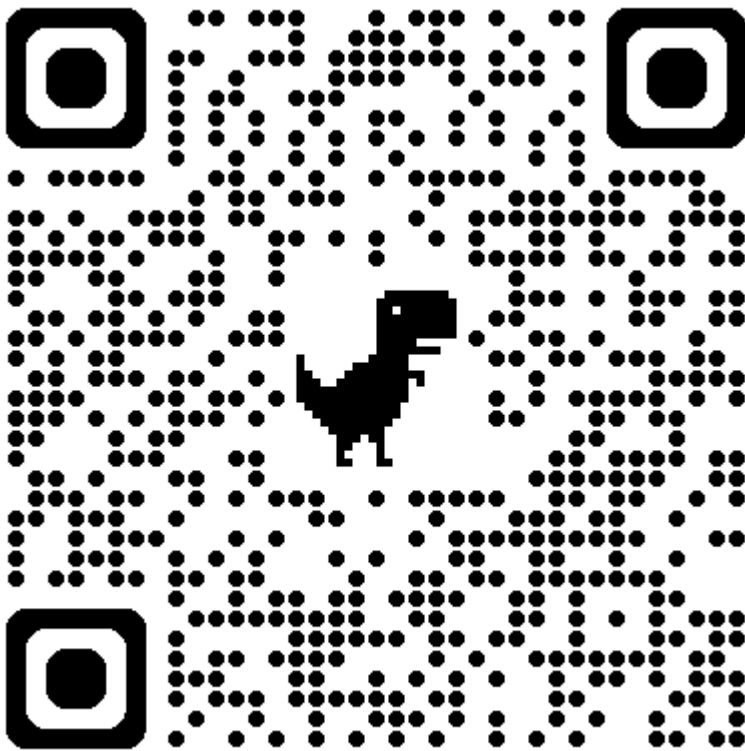
Fall 2025

Nat Trask

University of Pennsylvania

Supervised learning

Automatic differentiation





Supervised Learning

Fitting Models to data

Automatic Differentiation

Get the derivatives we need for
Newton-like methods

Last time: Supervised learning

Goal: Given data points (x_i, y_i) , find parameters θ that best fit a model

The Problem

- Model: $\hat{y} = f(x; \theta)$ (prediction)
- Data: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ (observations)
- Loss function: $L(\theta) = \sum_{i=1}^n (y_i - f(x_i; \theta))^2$ (error)

Optimization Approach:

- 1 Minimize loss: find θ^* where $\frac{\partial L}{\partial \theta} = 0$
- 2 **This is root finding!** Apply Newton's method
- 3 Or use optimization form: $\theta_{n+1} = \theta_n - \frac{\partial L / \partial \theta}{\partial^2 L / \partial \theta^2}$

Connection to Engineering:

- Fit helicopter control model to flight data
- Calibrate COVID model (β, γ) to case counts
- Train neural networks (gradient descent is simplified Newton)

A familiar example: polynomial regression

The Setup:

- **Training data:**

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

- **Model:**

$$\hat{y} = a_0 + a_1x + a_2x^2 + \dots + a_dx^d$$

- **Goal:** Find coefficients

$$\mathbf{a} = [a_0, a_1, \dots, a_d]$$

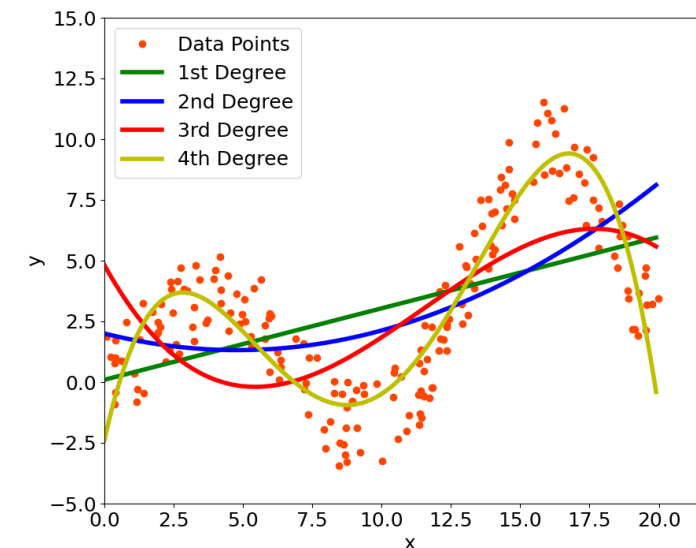
Loss Function (Mean Squared Error):

$$L(\mathbf{a}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

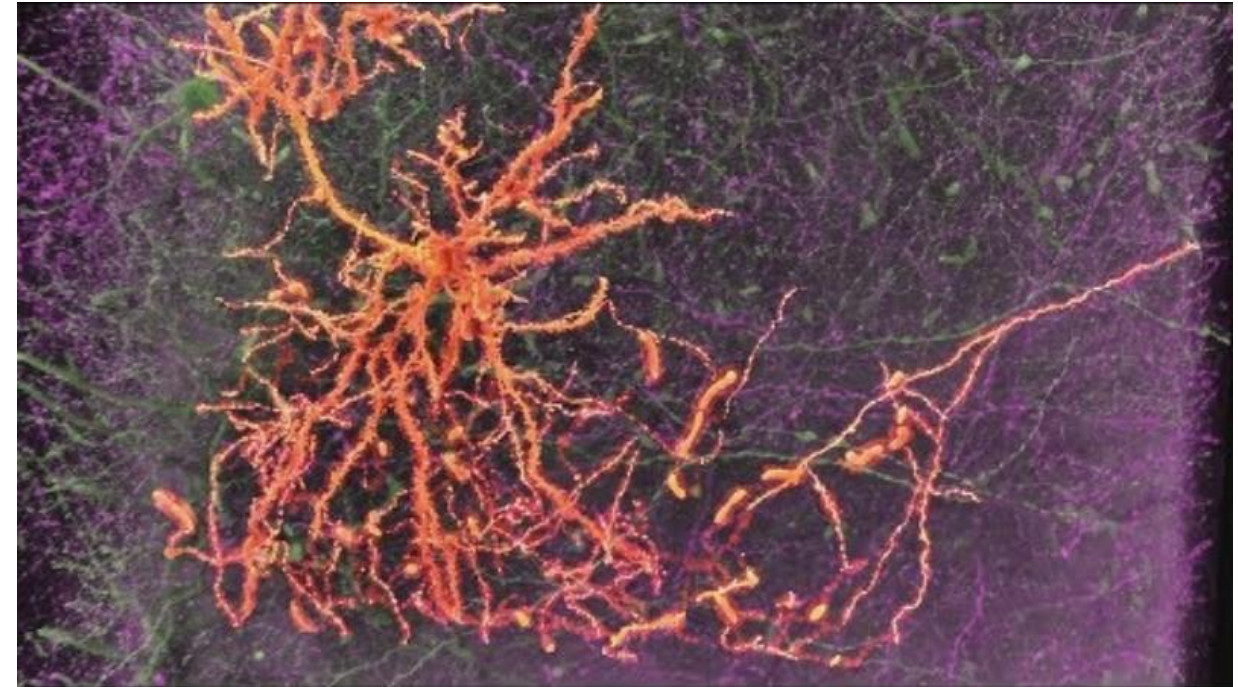
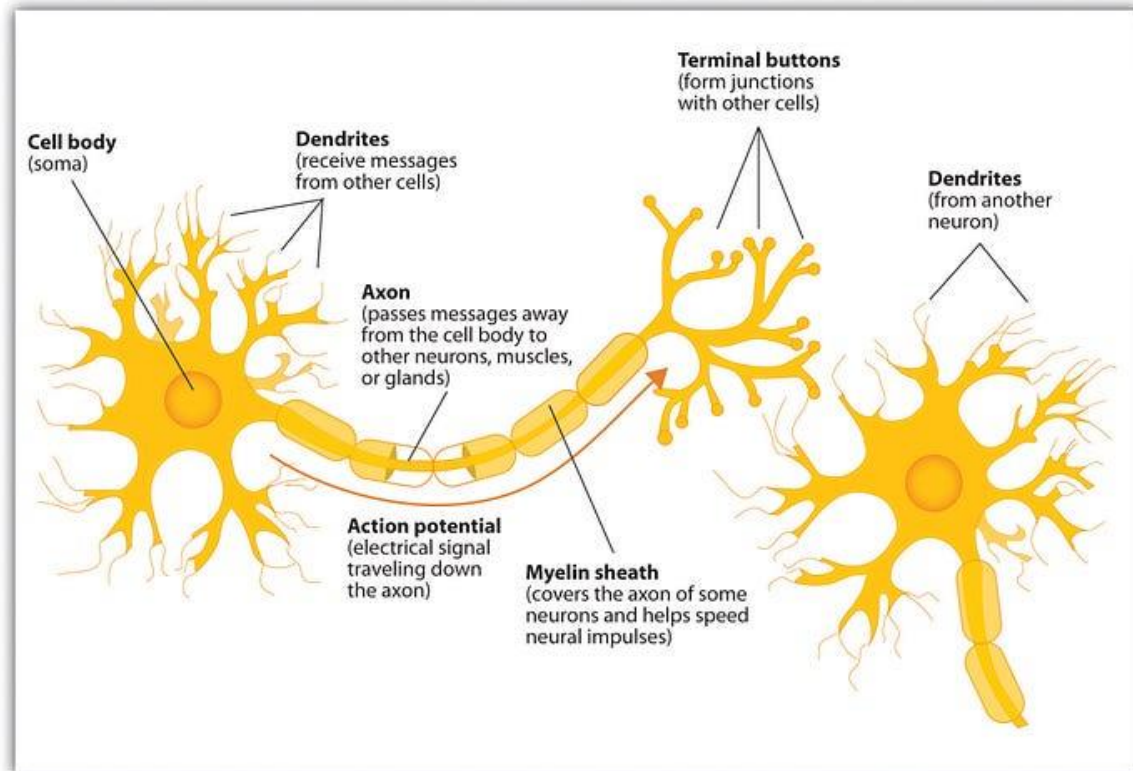
$$= \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^d a_j x_i^j \right)^2$$

This is Supervised Learning because:

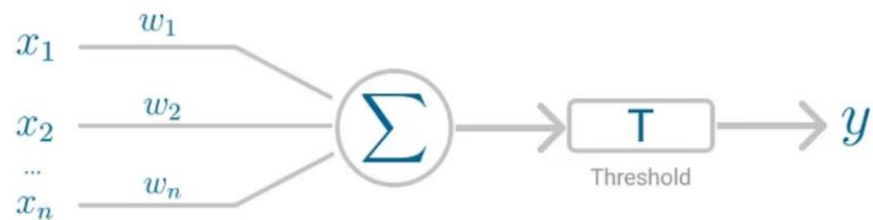
- 1 We have **labeled** training data
- 2 We learn a **function**
 $f(x) \rightarrow y$
- 3 We **minimize prediction error**
- 4 We can make **predictions** on new x



The full complexity of real-life neurons

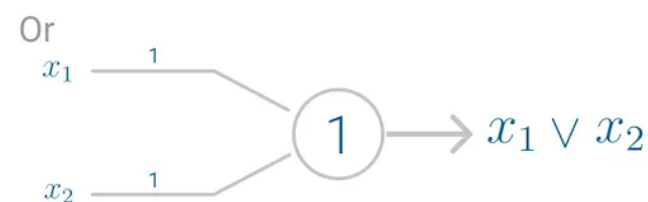
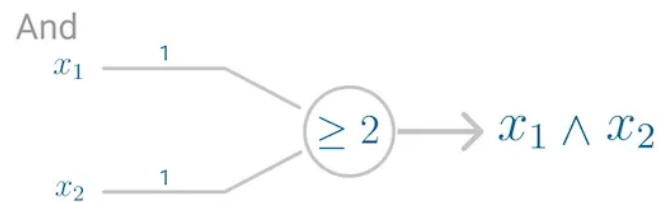


The multilayer perceptron (MLP) and artificial neurons



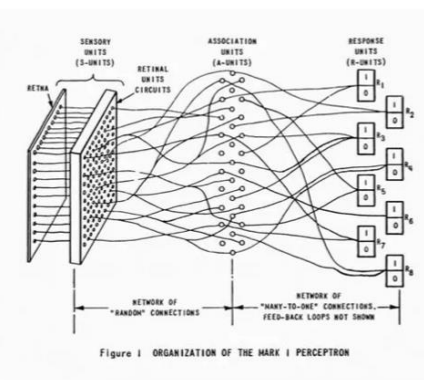
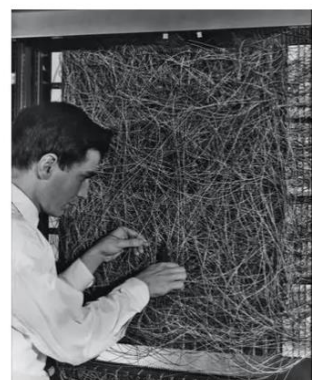
$$y = \begin{cases} 1, & \text{if } \sum_i w_i x_i - T > 0 \\ 0, & \text{otherwise} \end{cases}$$

weighted sum
threshold



The MLP circuit can implement simple logic operations like AND/OR

In 1943, [Warren McCulloch](#) and [Walter Pitts](#) proposed the binary [artificial neuron](#) as a logical model of biological neural networks.

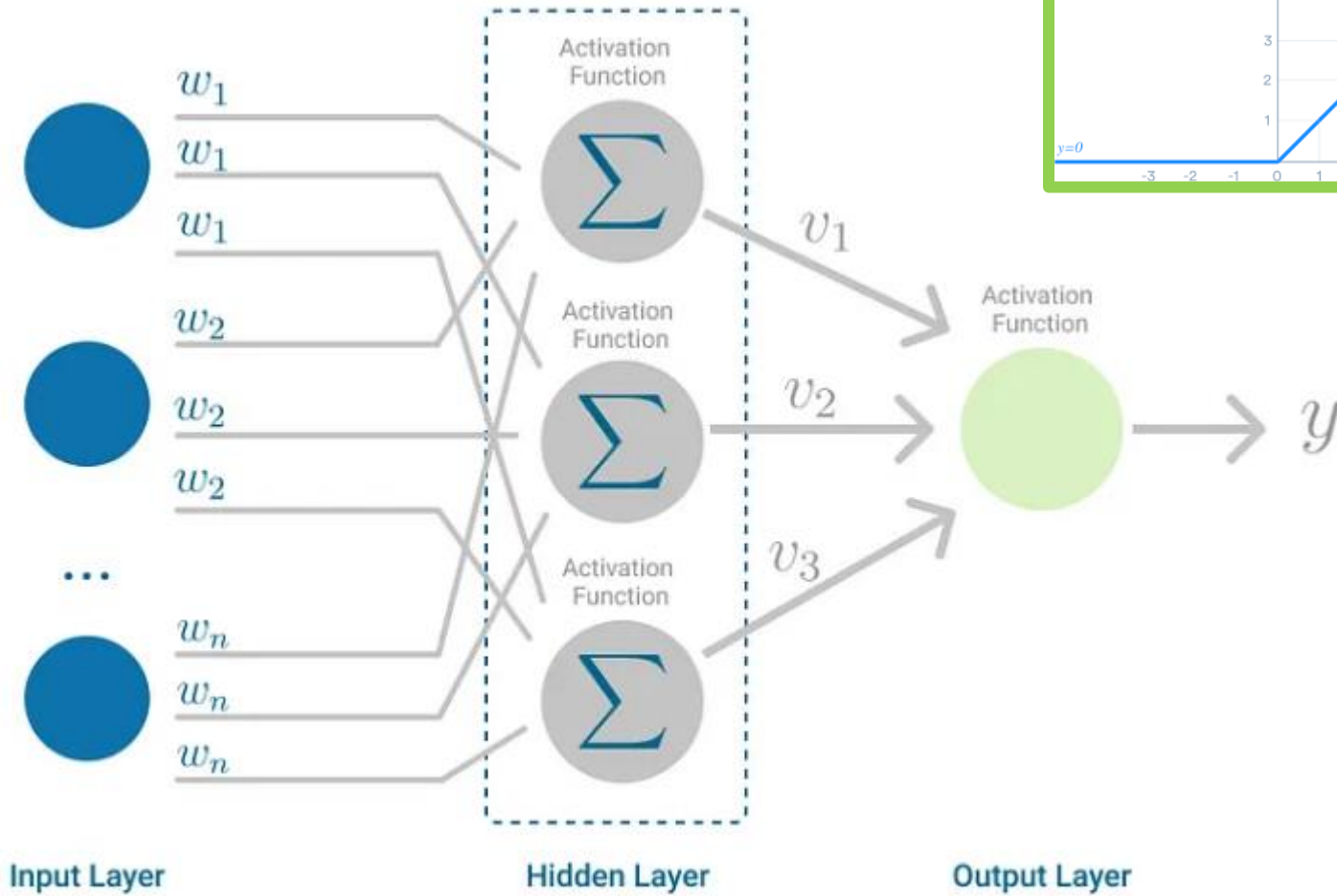


The Mark I Perceptron was a machine that implemented the perceptron algorithm for image recognition.



1. Feedforward | Mean Squared Error (MSE) computed

2. Backpropagation | Gradient is computed



The math that justifies using this for some wild predictions

Theorem (Cybenko)

Let σ be any continuous discriminatory function.

Then finite sums of the form

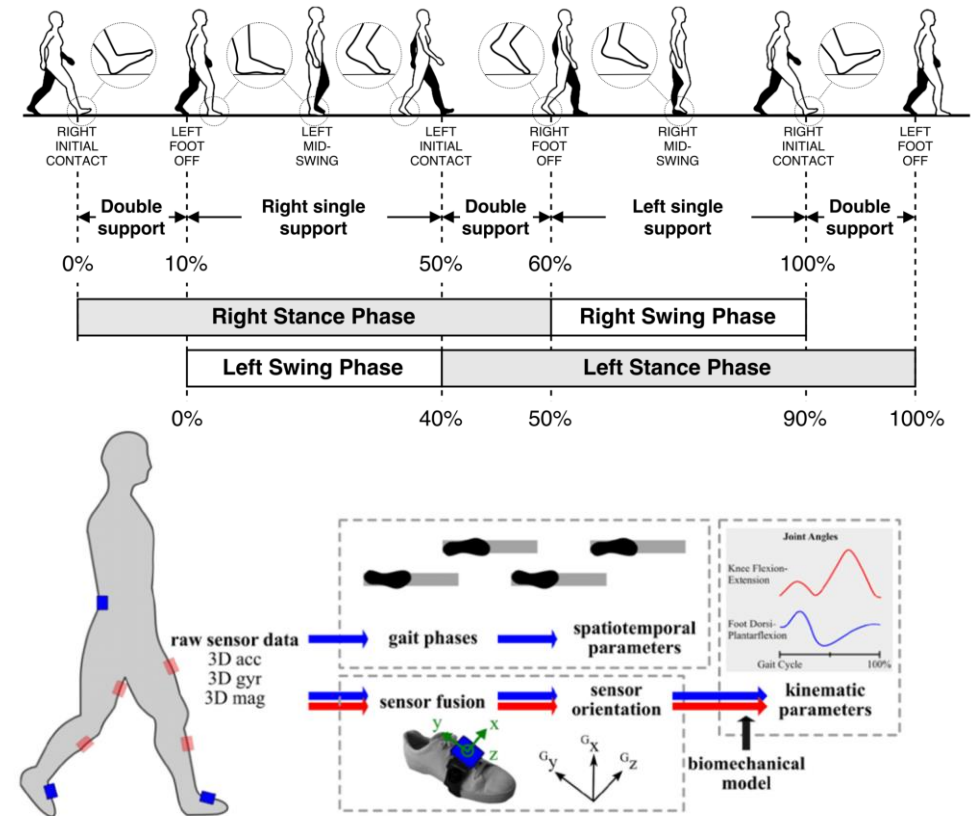
$$G(x) = \sum_{j=1}^N \alpha_j \sigma(w_j^T x + b_j), \text{ where } w_j \in \mathbb{R}^n, \alpha_j, b_j \in \mathbb{R}$$

are dense in $C(I_n)$.

In other words, given any $\varepsilon > 0$ and $f \in C(I_n)$, there is a sum $G(x)$ of the above form such that

$$|G(x) - f(x)| < \varepsilon, \quad \forall x \in I_n$$

In plain English: MLPs are capable of finding maps from *any* input to *any* output



The history leading up to where we're at today

In 1943, [Warren McCulloch](#) and [Walter Pitts](#) proposed the binary [artificial neuron](#) as a logical model of biological neural networks.^[11]

In 1958, [Frank Rosenblatt](#) proposed the multilayered [perceptron](#) model, consisting of an input layer, a hidden layer with randomized weights that did not learn, and an output layer with learnable connections.^[12]

In 1962, Rosenblatt published many variants and experiments on perceptrons in his book *Principles of Neurodynamics*, including up to 2 trainable layers by "back-propagating errors".^[13] However, it was not the backpropagation algorithm, and he did not have a general method for training multiple layers.

In 1965, [Alexey Grigorevich Ivakhnenko](#) and Valentin Lapa published [Group Method of Data Handling](#). It was one of the first [deep learning](#) methods, used to train an eight-layer neural net in 1971.^{[14][15][16]}

In 1967, [Shun'ichi Amari](#) reported^[17] the first multilayered neural network trained by [stochastic gradient descent](#), was able to classify non-linearly separable pattern classes. Amari's student Saito conducted the computer experiments, using a five-layered feedforward network with two learning layers.^[16]

[Backpropagation](#) was independently developed multiple times in early 1970s. The earliest published instance was [Seppo Linnainmaa](#)'s master thesis (1970).^{[18][19][16]} [Paul Werbos](#) developed it independently in 1971,^[20] but had difficulty publishing it until 1982.^[21]

In 1986, [David E. Rumelhart](#) et al. popularized backpropagation.^{[22][23]}

In 2003, interest in backpropagation networks returned due to the successes of [deep learning](#) being applied to [language modelling](#) by [Yoshua Bengio](#) with co-authors.^[24]

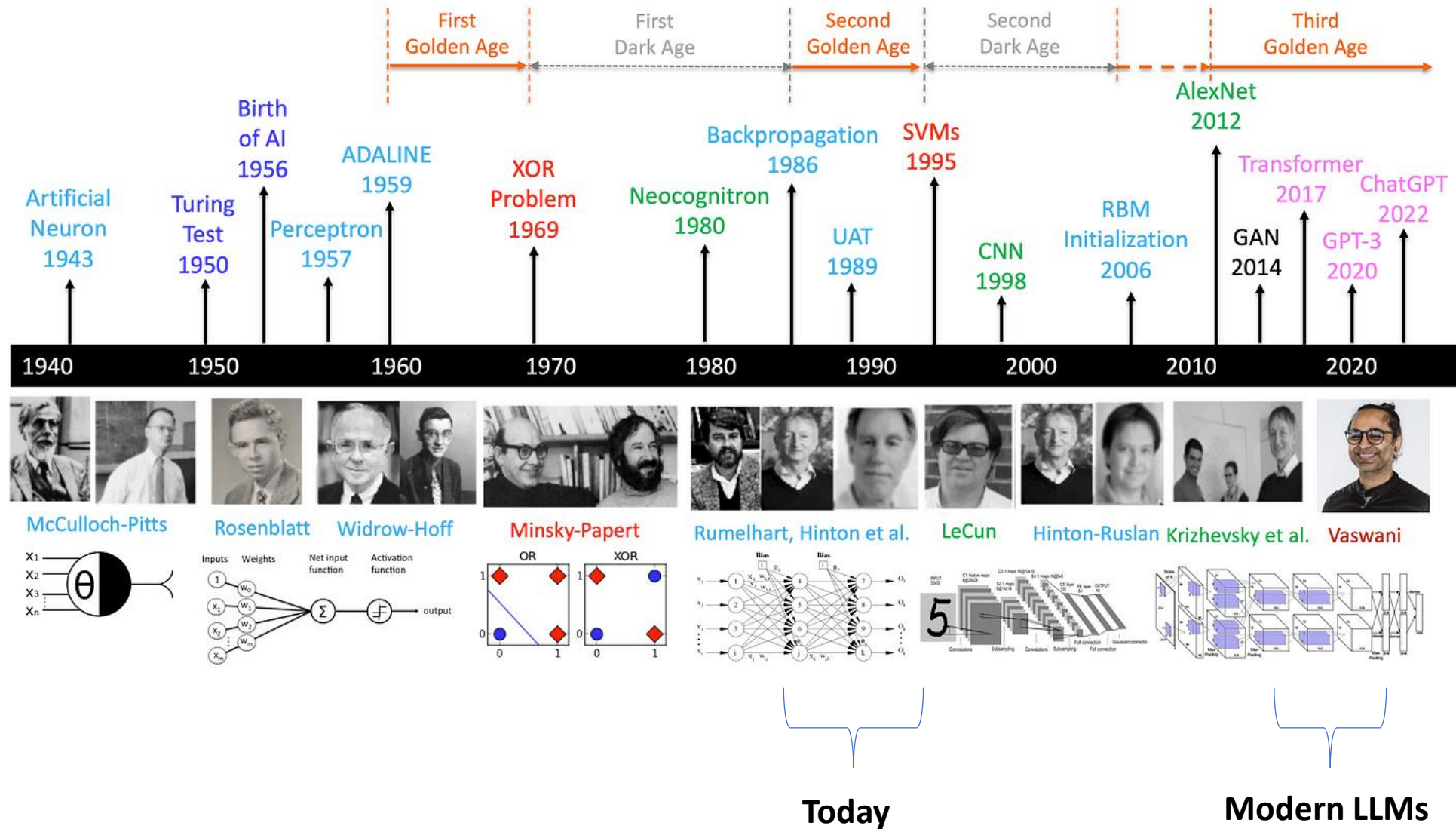
Automatic
gradients



Tensorflow



A Brief History of AI with Deep Learning



PyTorch is a backprop engine (aka an automatic derivative calculator)

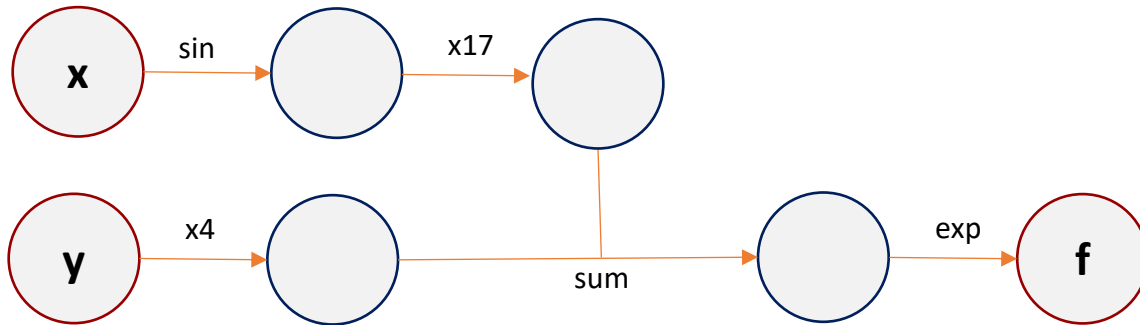
```
import torch
```

```
x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(4.0, requires_grad=True)
f = torch.exp(17 * torch.sin(x) + 4*y)
```



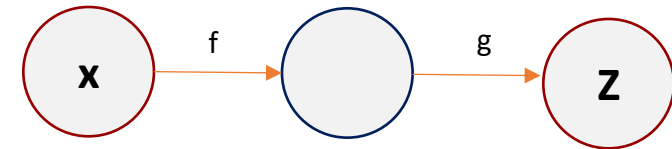
0.0s

Python



Step 1. Torch decomposes functions into a *computational graph*

$$z = f(g(x))$$



$$\frac{dz}{dx} = f'(g(x)) \cdot g'(x)$$

Step 2. Traversing the graph backward with the chain rule gives gradients

Syntax: Four step cookbook to take a gradient

Step 1: Define variables, set requires_grad=True

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True) # A vector  
y = torch.tensor(3, requires_grad=True) # A scalar
```

Step 2: Evaluate forward pass of function to differentiate

```
f = x**2 + y * x # Some function of x and y
```

Step 3: Evaluate backward() on output to compute gradients

```
f.backward() # Compute gradients
```

Step 4: Access dfdx and dfdy gradients via .grad attribute

```
print(x.grad) # Gradient of f w.r.t. x  
print(y.grad) # Gradient of f w.r.t. y
```

Warning: Need to run `x.grad.zero_()` before next `backward()`

An example computing a gradient – a forward and backward pass on the graph

```
# Build a variable x and set requires_grad=True to track computation with it
x = torch.tensor(3.0, requires_grad=True)
print("x =", x)

# Add a single node of computational graph: y = x^2
y = x**2
print("y = x^2 =", y)

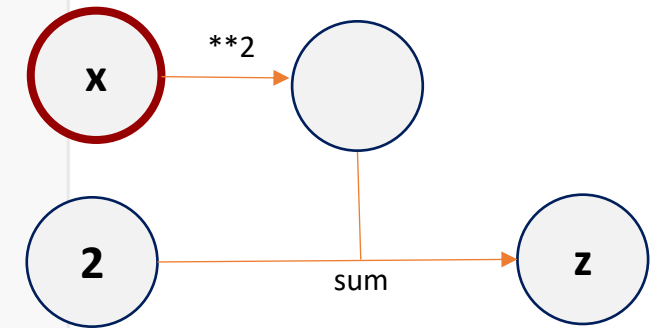
# Add a second node to the computational graph: z = y + 2
z = y + 2
print("z = y + 2 =", z)

# Compute the derivative dz/dx
z.backward() # This computes the derivative
print("dz/dx at x=3:", x.grad)

# Mathematical check: if z = x^2 + 2, then dz/dx = 2x = 2*3 = 6 ✓
```

✓ 2.6s

Python



```
x = tensor(3., requires_grad=True)
y = x^2 = tensor(9., grad_fn=<PowBackward0>)
z = y + 2 = tensor(11., grad_fn=<AddBackward0>)
dz/dx at x=3: tensor(6.)
```

Gradient descent: a quick and dirty Newton's method

Recall the update for Newton's method:

$$x_{n+1} = x_n - \underbrace{\frac{1}{f''(x_n)}}_{\text{step distance}} \underbrace{f'(x_n)}_{\text{step direction}}$$

Computing second derivatives is expensive; if we have n variables, it costs n^2 to compute the gradient of every gradient.

To make this efficient, gradient descent introduces a fixed step size α :

$$x_{n+1} = x_n - \alpha f'(x_n)$$

 **Compute with automatic differentiation!**

Step size. α must be tuned to ensure convergence. Too large can overshoot; too small leads to slow progress.

Today: use gradient descent to do supervised learning

Problem 1: Linear Regression

- Data: (x_i, y_i)
- Model: $\hat{y} = mx + b$
- Loss:
$$L(m, b) = \sum (y_i - (mx_i + b))^2$$
- Goal: Find m^*, b^* minimizing L

Problem 2: Binary Classification

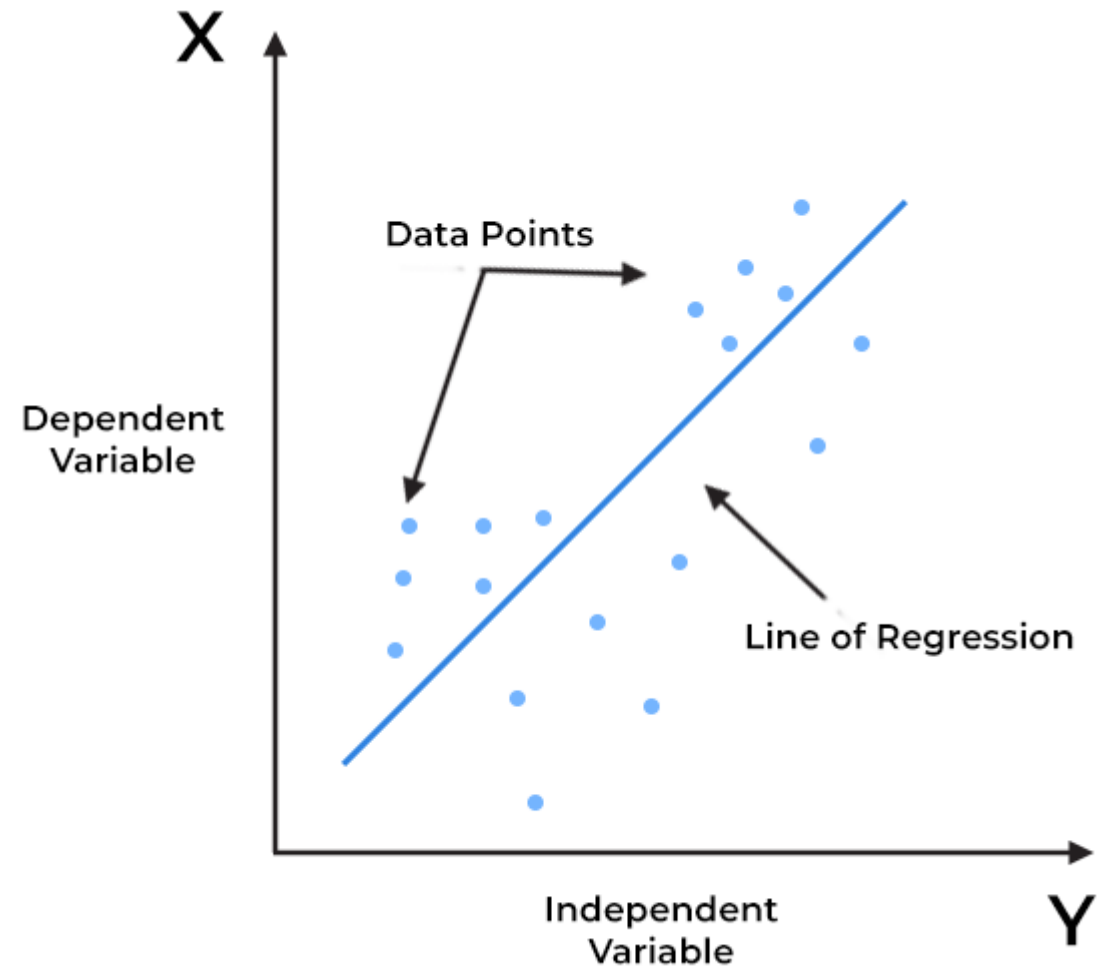
- Data: (x_i, y_i) with $y_i \in \{0, 1\}$
- Model: $\hat{y} = \sigma(wx + b)$
- Loss:
$$L(w, b) = - \sum y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$
- Goal: Find w^*, b^* minimizing L

Here $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function mapping to probabilities.

Today: use gradient descent to do supervised learning

Problem 1: Linear Regression

- Data: (x_i, y_i)
- Model: $\hat{y} = mx + b$
- Loss:
$$L(m, b) = \sum (y_i - (mx_i + b))^2$$
- Goal: Find m^*, b^*
minimizing L



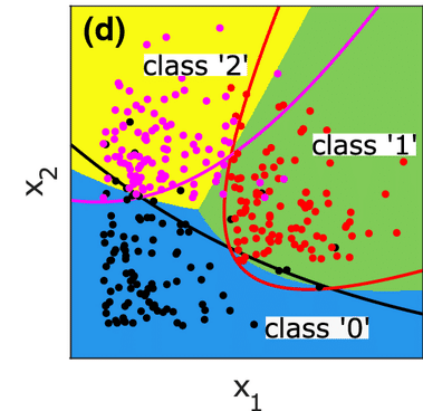
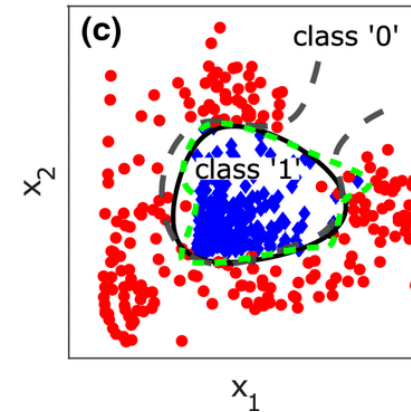
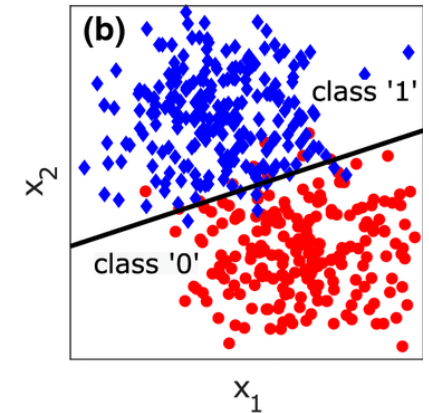
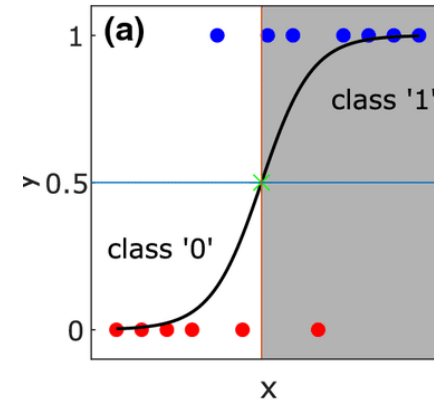
Today: use gradient descent to do supervised learning

Problem 2: Binary Classification

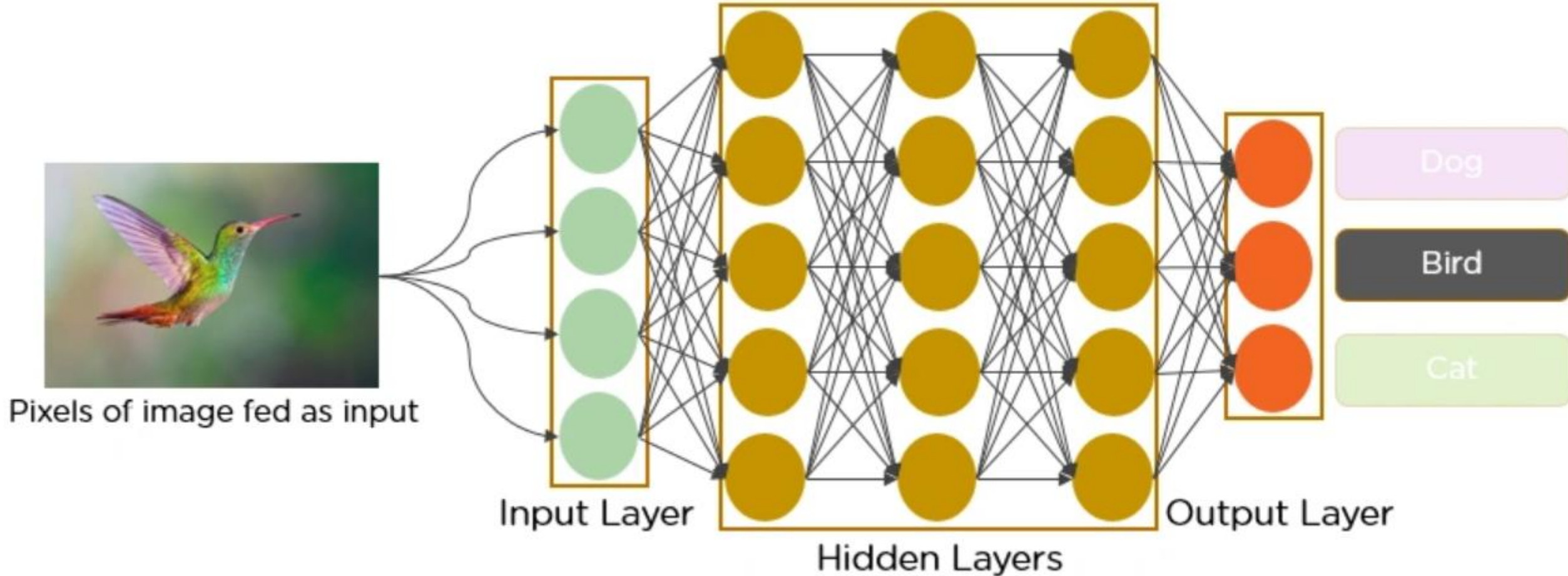
- Data: (x_i, y_i) with $y_i \in \{0, 1\}$
- Model: $\hat{y} = \sigma(wx + b)$
- Loss:

$$L(w, b) = - \sum y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

- Goal: Find w^*, b^* minimizing L

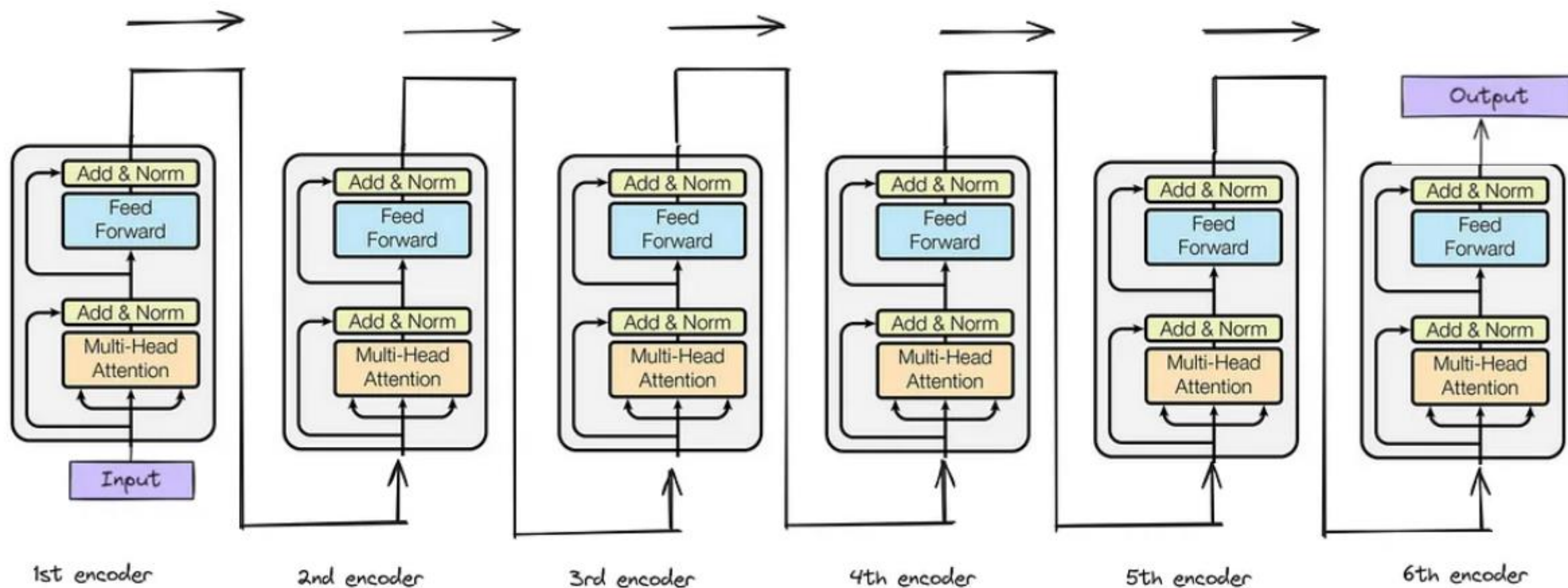


What's coming next week: *deep* learning



Repeated MLPs – typically
many more than shown here

What's coming next week: *beyond MLPs*



ENGR1050 Final Project Part Ordering F

If you intend to do a Pico based project, please fill out this form to order your parts. Orders must be received by the end of day Wednesday to receive on time.

- The budget per group is \$25/student
- Receiving parts on time will require providing a vendor with a fast delivery time. Digikey and Newark are preferred vendors.
- Before ordering a part, check the estimated delivery time is prior to Dec 3rd.
- Check with Dr. Trask (on Ed is fine) if you are getting something esoteric outside of the standard parts in class.
- You are responsible for doing research ahead of time to ensure you have appropriate skills to use the part (e.g. no complicated wiring diagrams, no soldering, no 3D printing etc)
- Orders that don't meet requirements will be rejected

To keep things simple, use low tech solutions where possible (e.g. rather than 3D printing a LCD display with mounts for buttons, you can attach to a piece of cardboard). This will keep complexity on the important bits (wiring and programming) and save you time.

